Herald

Herald — Operator Credentials Guide

Field	Value
Revision	3
Created	2026-05-21
Last modified	2026-05-31
Status	active
Status summary	r3: corrected the stale "Slack — planned, V2 / NOT YET IMPLEMENTED" section to **HRD-115 (adapter LIVE; live round-trip operator-cred-gated)** with the REAL env vars verified from source (`HERALD_SLACK_BOT_TOKEN` / `HERALD_SLACK_APP_TOKEN` / `HERALD_SLACK_CHANNEL_ID` — the planning-draft `HERALD_SLACK_SIGNING_SECRET` / `HERALD_SLACK_DEFAULT_CHANNEL` names were superseded by the shipped Socket-Mode adapter); cross-linked the new deep operator guides `messengers/SLACK.md` (Slack credential walkthrough) + `MTPROTO.md` (MTProto user-account login for the §11.4.98 harness). Prior r2: documented the `HERALD_TGRAM_OPERATOR_USERNAME` operator env var (Telegram env table row + Step 4b) per the participant/attribution contract `docs/design/PARTICIPANT_ATTRIBUTION.md` — designates the operator (env, not DB flag), drives `created_by`/`assigned_to` attribution + the @-tagging no-self-ping rule, generalizes to `HERALD_<CHANNEL>_OPERATOR_USERNAME`. Comprehensive step-by-step guide for obtaining and configuring every environment variable Herald requires — covers live integrations (Postgres, Redis, Telegram, Claude Code) AND reserved env-var names for planned integrations (Slack, Email, Max, Microsoft Teams, Lark, Discord, WhatsApp, Viber, OpenCode, Aider, Gemini, Cursor). Documents the dual-source resolution model (shell exports vs `.env`) per spec §3.3 + Universal Constitution §11.4.10.
Issues	none
Issues summary	—
Fixed	(n/a — new guide)
Continuation	bump when new channel HRDs land (HRD-NNN Slack live → add detailed Slack section; HRD-NNN Email live → SMTP/Resend sections; etc.)

Table of contents

Per-service dedicated guides

This umbrella document covers the credentials model, resolution order, audit checklist, and per-service reserved env-var names. For **detailed step-by-step setup** of each integration, see the dedicated per-service guides:

Messengers — under `messengers/`

- `messengers/TELEGRAM.md` — **LIVE** (HRD-011, bot setup); deep operator guide `TELEGRAM.md`
- `MTPROTO.md` — **MTProto user-account login** (the §11.4.98 full-automation harness; `qaherald mtproto login`) — see also the §"MTProto user-account harness" section below
- `messengers/SLACK.md` — **HRD-115** (adapter LIVE; live round-trip cred-gated)
- `messengers/EMAIL.md` — planned V2 (SMTP + Resend)
- `messengers/MAX.md` — planned V2
- `messengers/TEAMS.md` — planned V3
- `messengers/LARK.md`, `DISCORD.md`, `WHATSAPP.md`, `VIBER.md` — planned later iterations

LLM / agent dispatchers — under `dispatchers/`

- `dispatchers/CLAUDE_CODE.md` — **LIVE** (HRD-012)
- `dispatchers/OPENCODE.md`, `AIDER.md`, `GEMINI.md`, `CURSOR.md`, `ANTHROPIC.md` — planned

Resolution order (12-factor)

Herald follows 12-factor conventions (spec V3 §3.3 + §11.4.10) with a strict, single-direction precedence. From highest to lowest:

1. **Explicit CLI flag** — e.g. `pherald migrate up --dsn=postgres://...` always wins.
2. **Shell-exported env var** — read by Herald's Go code via `os.Getenv`. Source: your `~/.bashrc`, `~/.zshrc`, your CI's secret store, a Kubernetes `ConfigMap` / `Secret`, the parent shell that launched `pherald`, etc.
3. **Project-local `.env` file** — fallback only. Read automatically by `docker-compose` for the quickstart stack; for native `pherald` execution you source it explicitly (see below).
4. **Compiled default** — lowest precedence. Examples: `HERALD_DB_PORT=24100`, `HERALD_REDIS_DB=0`, `HERALD_CLAUDE_BIN=claude`.

The load-bearing rule: shell exports ALWAYS win over `.env`. This means an operator can have team defaults in `.env` while still overriding per-developer-host via `.bashrc`. The override never accidentally leaks back into `.env`. This is exactly the contract Herald's `commons_infra.envOr()` helper (`commons_infra/boot.go`) enforces: it calls `os.Getenv` first; if that returns empty, it falls back to a compiled default. The `.env` is read at one layer above — either by `docker-compose` or by the operator's shell when they `source .env`.

Setting credentials in `~/.bashrc` / `~/.zshrc` (shell-export path)

Append `export` lines to your shell's startup file. The exact file depends on your default shell:

- **macOS default (zsh)**: `~/.zshrc`
- **Linux default (bash)**: `~/.bashrc` (interactive non-login) or `~/.bash_profile` (login)
- **Both shells**: `~/.profile` (POSIX fallback — sourced by both bash login shells and zsh)

Example block to add (substitute real values from later sections of this guide):

```
# Herald — Postgres
export HERALD_DB_PASSWORD='your-strong-postgres-password'
# (other HERALD_DB_* vars are fine at defaults for quickstart
        compose)

# Herald — Redis
export HERALD_REDIS_PASSWORD='your-strong-redis-password'

# Herald — Telegram (HRD-011)
export HERALD_TGRAM_BOT_TOKEN='1234567890:AAH1lab2c3d4e5f6g7h8i9j0kK_LmN3o
export HERALD_TGRAM_CHAT_ID='-1001234567890'

# Herald — Claude Code (HRD-012)
export HERALD_CLAUDE_PROJECT_NAME='Herald'
```

```
# HERALD_CLAUDE_BIN defaults to 'claude' (looked up via $PATH).
# HERALD_CLAUDE_SESSION_UUID is auto-bootstrapped on first run;
        set only for testing.
```

After editing, run `source ~/.zshrc` (or `~/.bashrc`) or open a new terminal so the changes take effect.

Verify:

```
echo "${HERALD_TGRAM_BOT_TOKEN:0:10}…"   # prints first 10 chars
        then ellipsis
# Should NOT print the empty string. If it does, the export
        didn't take.
```

Use this path for credentials that follow you across all projects + checkouts (your personal Claude session UUID; your workstation-wide Telegram bot token; etc.).

Setting credentials in `.env` (project-local path)

For credentials specific to one Herald checkout (a per-deploy Postgres password, a per-environment Slack webhook), use `.env`:

```
cd /Users/milosvasic/Projects/Herald
cp quickstart/.env.example .env
chmod 600 .env                  # readable only by you —
        discourages accidental cat
$EDITOR .env                    # fill in real values
```

The `.env` file is git-ignored (verified via `.gitignore` line 28: `.env`). Never `git add .env`. Never paste real credentials into a PR description, commit message, Slack channel, or any artifact that gets persisted.

To consume `.env` from **native** `pherald` (i.e. NOT through `docker-compose`), you must source it into your shell first:

```
set -a                          # auto-export every variable
        assigned hereafter
source .env
set +a                          # turn auto-export back off
pherald serve                   # now sees all .env vars
```

Or use direnv if you have it: `echo 'dotenv' > .envrc; direnv allow`. With `direnv`, every `cd` into the repo automatically exports the `.env` vars; every `cd` out unsets them.

For **compose-based** runs (`docker-compose -f quickstart/docker-compose.quickstart.yml up`): `docker-compose` (and `podman-compose`) reads `.env` automatically from the project root. No sourcing required.

Use this path for credentials specific to one Herald checkout or one deploy.

Constitutional rules — what NEVER goes in git

Per Helix Universal Constitution §11.4.10 + Herald §107:

- **NEVER** commit a `.env` file (anywhere — repo root, subdirectories, anywhere). Verify with `git ls-files | grep -E '^\.env$'` — must be empty.
- **NEVER** commit real bot tokens, API keys, passwords, OAuth secrets, session UUIDs.
- **NEVER** put credentials in commit messages, PR descriptions, `docs/Issues.md`, `docs/Fixed.md`, `CHANGELOG`, `README`, scripts, or any tracked artifact.
- **DO** commit `quickstart/.env.example` with placeholder values (`change-me-...`, `re_xxxxxxxxxxxxxxxxxxxxxxxxxxxx`, etc.) — the file's purpose is documentation, not secret storage.
- **DO** commit code that READS env vars via `os.Getenv` — the code is harmless without the vars set.

A leaked credential in tracked history is a §107 release-blocker:

- If you discover one in your local working tree, do NOT commit; remove it first.
- If you discover one already pushed to a remote, **rotate the credential immediately** (regenerate the token, change the password, etc.) — git history scrubbing is forensic-quality but does not retroactively invalidate the leaked secret. Inform the operator.

Per-service credential setup

Postgres (LIVE)

REQUIRED — Herald cannot run without these. Closed by HRD-010.

Env var	Description	Default	Where to obtain
`HERALD_DB_HOST`	Postgres host	`127.0.0.1`	quickstart compose / your own deploy
`HERALD_DB_PORT`	Postgres port	`24100`	matches quickstart compose mapping `24100:5432`
`HERALD_DB_USER`	DB role for Herald migrations	`herald`	created by quickstart compose's `POSTGRES_USER`
`HERALD_DB_PASSWORD`	DB password for that role	(no default)	**YOU choose** — `openssl rand -base64 32`
`HERALD_DB_NAME`	Database name	`herald`	matches `POSTGRES_DB`
`HERALD_PG_DSN`	URL form used by `pherald migrate` CLI	(composed from above)	`postgres://herald:<pass>@host:port/herald`

Setup steps:

1. Choose a strong password: `openssl rand -base64 32` (copy output).
2. Add to `.env`:

   ```
   HERALD_DB_PASSWORD=<paste-the-output>
   ```

3. Or export from shell:

```
export HERALD_DB_PASSWORD='<paste-the-output>'
```

4. Boot Postgres via compose: `docker-compose -f quickstart/docker-compose.quickstart.yml up -d postgres`.
5. Verify: `pherald migrate status` → should print `schema version: 0` on a fresh DB.
6. Apply migrations: `pherald migrate up` → should print `applied 11 migration(s)`.

Redis (LIVE)

REQUIRED. Closed by HRD-010 (Redis health-check live; full feature wiring in subsequent HRDs).

Env var	Description	Default	Notes
`HERALD_REDIS_ADDR`	host:port	`127.0.0.1:24200`	matches quickstart compose `24200:6379`
`HERALD_REDIS_PASSWORD`	ACL "default" user password	(no default)	**YOU choose**
`HERALD_REDIS_DB`	DB index	`0`	default OK for single-tenant

Setup mirrors Postgres: strong random password to `.env` or shell. Verify with the integration test `TestUp_PopulatesRedis_TTLRoundTrip` (set the env var first; run `go test ./commons_infra/ -tags=integration -run TestUp_PopulatesRedis_TTLRoundTrip -count=1`).

Telegram — HRD-011 (LIVE)

REQUIRED for the Telegram channel. Closed by HRD-011 (code complete; live evidence requires operator credentials).

Env var	Description	Default	How to obtain
`HERALD_TGRAM_BOT_TOKEN`	Bot API token	(no default)	@BotFather (see steps below)
`HERALD_TGRAM_CHAT_ID`	Target chat ID (numeric)	(no default)	`/getUpdates` (see steps below)
`HERALD_TGRAM_LIVE_INBOUND`	Enables the E19 vertical-slice test	(unset)	set to `1` when running E19 + hand-sending a message
`HERALD_TGRAM_WEBHOOK_SECRET`	Webhook secret_token (PRODUCTION)	(unset)	`openssl rand -hex 32` — reserved for HRD-NNN webhook-ingress
`HERALD_TGRAM_OPERATOR_USERNAME`	The operator's Telegram `@username` — used for workable-	(unset)	your own Telegram `@username` (e.g. `@milos85vasic`) — see step below

Env var	Description	Default	How to obtain
	item attribution + @-tagging		

Step 1: Create a bot via @BotFather

1. Open Telegram → search `@BotFather` → start chat.
2. Send `/newbot`.
3. Choose a display name (e.g. "Herald Operator Bot").
4. Choose a username ending in `bot` (e.g. `herald_operator_bot`).
5. BotFather returns a token of the form `1234567890:AAH1lab2c3d4e5f6g7h8i9j0kK_LmN3oPqR4S5t`.
6. **Copy the token immediately** — set `HERALD_TGRAM_BOT_TOKEN=<the-token>`. (BotFather will let you regenerate if you lose it; never paste the token publicly.)

Step 2: Get your chat ID

1. Add the bot to a chat (or DM it directly), then send any message to it.
2. Visit `https://api.telegram.org/bot<your-token>/getUpdates` in a browser (substitute the real token).
3. Look in the JSON response for `"chat":{"id": NUMBER, ...}`. That NUMBER is your chat ID.
    - For groups, the ID is negative (e.g. `-1001234567890`).
    - For DMs, the ID is positive and equals the user ID.
4. Set `HERALD_TGRAM_CHAT_ID=<the-number>`.

Step 3: (Optional) Enable live-inbound for E19 vertical-slice test

Set `HERALD_TGRAM_LIVE_INBOUND=1` ONLY when you intend to hand-send a Telegram message during a test run so the `Subscribe` loop actually pulls an inbound update. Leave unset for everyday operation — the long-poll goroutine still runs but the E19 test SKIPs without this flag.

Step 4: (Future — webhook mode) generate a webhook secret

For production, prefer webhook over long-poll for lower latency:

```
openssl rand -hex 32     # generate a 256-bit secret
```

Set `HERALD_TGRAM_WEBHOOK_SECRET=<the-hex>`. Configure the bot's webhook URL to `https://your-herald.example.com/v1/webhooks/tgram` (when HRD-NNN ships webhook ingress; long-poll only in v1).

Step 4b: (Attribution + @-tagging) set the operator username

Per the participant/attribution contract (docs/design/PARTICIPANT_ATTRIBUTION.md), set `HERALD_TGRAM_OPERATOR_USERNAME` to your own Telegram `@username`:

```
export HERALD_TGRAM_OPERATOR_USERNAME='@milos85vasic'
```

This designates you as the **operator** (the one human who drives the system via the Claude Code CLI) — by env var, NOT a DB flag. Your canonical handle equals this value. It drives two behaviours:

- **Attribution.** Workable items opened via the Claude Code CLI prompt get `created_by` = this value; `assigned_to` defaults to it.
- **@-tagging.** The operator is NEVER @-tagged on notifications (no self-ping); only non-operator human assignees/openers are tagged (and `Claude`, the system agent, is never tagged).

The env var generalizes to `HERALD_<CHANNEL>_OPERATOR_USERNAME` for any other messenger (e.g. `HERALD_SLACK_OPERATOR_USERNAME`). A Participant may have a different `@username` on each messenger; the per-channel mapping lives in PG `subscriber_aliases.username`. See MESSENGER_CHANNELS.md §6A and WORKABLE_ITEMS_INTEGRATION.md §3.6–§3.8.

Step 5: Verify

```
go test ./commons_messaging/channels/tgram/ -tags=integration
        -run TestHealthCheck_LiveBotAPI -count=1 -timeout=60s
```

Expected: PASS — proves the token is valid + the bot is enabled.

Then run the E17 evidence test:

```
go test ./commons_messaging/channels/tgram/ -tags=integration
        -run TestSend_PersistsDeliveryEvidence -count=1
        -timeout=300s
```

Expected: PASS — a real Telegram message arrives in your configured chat + a row appears in `outbound_delivery_evidence`.

Claude Code dispatcher — HRD-012 (LIVE)

REQUIRED for the Claude Code LLM dispatch surface. Closed by HRD-012 (live PASS evidence captured: Tasks 6 + 7 produced 24s + 36s live runs against real `claude --resume`).

Env var	Description	Default	How to obtain
`HERALD_CLAUDE_BIN`	Path to `claude` CLI	`claude` (PATH lookup)	install Claude Code per Anthropic instructions
`HERALD_CLAUDE_PROJECT_NAME`	Identifier for the consuming project	(no default)	YOU choose — typically matches your repo folder name (e.g. `Herald`, `ATMOSphere`)
`HERALD_CLAUDE_SESSION_UUID`	(Optional) Pre-existing Claude session UUID	(auto-bootstrapped)	bootstrap path resolves this; set explicitly only for diagnostics
`HERALD_CLAUDE_WORKDIR`	(Optional) Working dir for `cmd.Dir` of	.	set if your consuming project is not the cwd

Env var	Description	Default	How to obtain
	`claude --resume`		

Step 1: Install Claude Code

Follow Anthropic's official Claude Code install guide. Verify with:

```
claude --version
```

Should print a version like `claude 1.x.y`. If `claude` isn't on `$PATH`, set `HERALD_CLAUDE_BIN=/absolute/path/to/claude`.

Step 2: Choose a project name

`HERALD_CLAUDE_PROJECT_NAME` is used as the anchor file name (e.g. `<workdir>/.herald/claude-code/sessions/Herald.session`). MUST be a valid filename — alphanumerics + dashes + underscores. Typically you choose your repo folder name.

```
export HERALD_CLAUDE_PROJECT_NAME='Herald'
```

Step 3: Bootstrap a session (two paths)

Path A — auto-bootstrap (the §33.2-canonical path):

Just omit `HERALD_CLAUDE_SESSION_UUID`. Herald's `ResolveSession()` will spawn `claude --print "Initializing Herald session for project: <name>"`, capture the new session UUID from stdout, persist it to `<workdir>/.herald/claude-code/sessions/<project>.session`, and reuse it across runs.

Path B — pre-create a session manually (for testing / diagnostics):

```
claude --print "init"
# Watch the session ID printed at the end of stdout — copy it.
export HERALD_CLAUDE_SESSION_UUID='3e67dcd3-c66f-4687-
        b936-562191437310'   # example
```

Step 4: Verify

```
go test ./commons_messaging/dispatch/claude_code/
        -tags=integration -run TestDispatch_LiveClaudeInvocation
        -count=1 -timeout=300s
```

Expected: PASS in ~20-30s — Claude receives an envelope, replies with the `<<<HERALD-REPLY>>>` JSON line, parsed with non-empty Outcome + Summary.

Slack — HRD-115 (adapter LIVE; live round-trip operator-cred-gated)

Status: the Slack channel adapter is **implemented and hermetically tested** (Wave 7 — `commons_messaging/channels/slack/`, **Socket Mode**). The live round-trip (HRD-115)

is gated only on operator-supplied credentials. **Full step-by-step credential-acquisition walkthrough → messengers/SLACK.md.** The adapter + live test read exactly these env vars (verified from source — the planning-draft `HERALD_SLACK_SIGNING_SECRET` / `HERALD_SLACK_DEFAULT_CHANNEL` names were superseded; the shipped adapter uses Socket Mode + `HERALD_SLACK_CHANNEL_ID`):

Env var	Description
`HERALD_SLACK_BOT_TOKEN`	`xoxb-...` bot token from the Slack app's "Install to Workspace" / OAuth flow
`HERALD_SLACK_APP_TOKEN`	`xapp-...` app-level token for **Socket Mode** (the inbound transport)
`HERALD_SLACK_CHANNEL_ID`	Target channel ID (e.g. `C012345ABCD`) — Slack channels are addressed by ID, not name

Tokens are §107-redacted out of every error message (`sanitizeError()` + `TestSlack_Send_ErrorDoesNotLeakToken` plants a secret and asserts its absence). Once the three are set in `.env`, prove the live round-trip with the **E127** / `TestSlack_Live_Send` gate — see messengers/SLACK.md §7; evidence lands under `docs/qa/HRD-115-LIVE-*/` per §107.x.

Email (SMTP) — planned, V2

Status: NOT YET IMPLEMENTED. Spec V3 §11.4. When SMTP channel lands:

Env var	Description
`HERALD_SMTP_HOST`	mail relay hostname
`HERALD_SMTP_PORT`	typically 587 (STARTTLS), 465 (TLS), or 25 (plain — discouraged)
`HERALD_SMTP_USERNAME`	mailbox username
`HERALD_SMTP_PASSWORD`	mailbox password OR app-password (per provider)
`HERALD_SMTP_FROM`	sender address (must be authorized for the username)

Email (Resend) — planned, V2

Status: NOT YET IMPLEMENTED. Alternative to raw SMTP. Spec V3 §11.4.

Env var	Description
`HERALD_RESEND_API_KEY`	`re_xxxxx...` from Resend dashboard (Settings → API Keys)
`HERALD_RESEND_FROM`	sender address verified in Resend

Max — planned, V2

Status: NOT YET IMPLEMENTED. Spec V3 §11.3. Russian-market messenger.

Env var	Description
`HERALD_MAX_BOT_TOKEN`	TBD — Max Bot API token format
`HERALD_MAX_CHAT_ID`	TBD

Detailed setup will follow when Max lands as an HRD.

Microsoft Teams — planned, V3

Status: NOT YET IMPLEMENTED. Spec marks Teams as a later iteration (post-V2).

Lark / Discord / WhatsApp / Viber — planned, later iterations

Per spec V3 §11, these are reserved for later iterations. Env var names will be defined when each HRD opens. Reserve a comment block in your `.env` for "later-iteration channels" to keep your file aligned with the spec.

Alternate LLM dispatchers (OpenCode / Aider / Gemini / Cursor / Managed Agents) — planned

Status: NOT YET IMPLEMENTED. Spec V3 §33 declares `Dispatcher` as a pluggable interface; only `claude-code` ships in V1. Future env vars (TBD HRDs):

Env var	Dispatcher
`HERALD_OPENCODE_*`	OpenCode
`HERALD_AIDER_*`	Aider
`HERALD_GEMINI_API_KEY`	Gemini Managed Agent
`HERALD_CURSOR_*`	Cursor
`ANTHROPIC_API_KEY`	Anthropic Managed Agent (Bedrock / Vertex variants TBD)

Quickstart compose vs native pherald

Two ways to run Herald:

Mode	Env source	Use case
Quickstart compose	`docker-compose -f quickstart/docker-compose.quickstart.yml up` reads `.env` automatically from the repo root + interpolates shell-exported vars on top	development, integration tests, demos
Native `pherald`	`pherald serve`, `pherald migrate` — `os.Getenv` only; reads ONLY shell-exported vars. Source `.env` first with `set -a; source .env; set +a` (or use `direnv`)	production-like, performance testing, CI

In compose mode, `.env` is the canonical source. In native mode, shell exports are. Both should reach the same end state per the resolution order at the top of this doc.

Audit checklist (run before every commit)

Before `git commit`, run:

```
cd /Users/milosvasic/Projects/Herald

# 1. .env is not tracked
git ls-files | grep -E '^\.env$' \
  && echo "BLOCK: .env is tracked — REMOVE BEFORE COMMIT" \
  || echo "OK: .env not tracked"
```

```
# 2. No obvious-format secrets in tracked files
git ls-files | xargs grep -lE "ghp_[a-zA-Z0-9]{36}|sk-[a-zA-Z0-9]
        {48}|xox[bps]-|AKIA[A-Z0-9]{16}|6[0-9]{9}:AAH[a-zA-
        Z0-9_-]{30,}" 2>/dev/null \
  | grep -v '\.env\.example' \
  && echo "BLOCK: probable secret in tracked files" \
  || echo "OK: no obvious-format secrets"

# 3. .env.example has only placeholder values (no real-looking
        secrets)
grep -E '=[a-zA-Z0-9+/=]{40,}' quickstart/.env.example \
  | grep -vE 'change-me|xxxxx|example|<.+>|FILLME' \
  && echo "WARN: .env.example may contain real values — verify
        each line" \
  || echo "OK: .env.example has only placeholder values"
```

Any "BLOCK" output means do NOT commit. Rotate the leaked credential, scrub from working tree, re-run the audit, and only then commit.

Troubleshooting

"connection refused" on Postgres: The compose stack isn't up. Run `docker-compose -f quickstart/docker-compose.quickstart.yml up -d postgres` and wait ~3s.

"SQLSTATE 28P01" (password authentication failed): The container's stored password hash doesn't match the env var. Either reset the volume (`podman-compose down -v` — destroys data) or run `ALTER USER herald PASSWORD '<your-env-var-value>';` inside the container.

`pherald migrate up` **says "HERALD_PG_DSN environment variable must be set"**: Compose-style env vars (`HERALD_DB_HOST/PORT/USER/PASSWORD/NAME`) are NOT automatically composed into `HERALD_PG_DSN`. Set the DSN explicitly:

```
export HERALD_PG_DSN="postgres://herald:${HERALD_DB_PASSWORD}
        @127.0.0.1:24100/herald"
```

Telegram integration test SKIPs even though token is set: Confirm both `HERALD_TGRAM_BOT_TOKEN` AND `HERALD_TGRAM_CHAT_ID` are set. The SKIP guard requires both.

`claude` **is on PATH but Dispatch test still SKIPs**: Confirm `HERALD_CLAUDE_PROJECT_NAME` AND (for the persistence test) `HERALD_CLAUDE_SESSION_UUID` are set. The SKIP guard layers all three checks.

".env values not picked up by native `pherald`**"**: You must source `.env` first — `pherald` reads `os.Getenv` only, not `.env` files. Use `set -a; source .env; set +a` or `direnv`.

"My credential leaked into git history": Rotate the credential IMMEDIATELY (regenerate the token, change the password). Then escalate per Universal Constitution §11.4.10 — git-history

scrubbing is forensic-quality only; the leaked value is forever compromised once pushed to a remote.

MTProto user-account harness (Wave 8 Track B — REQUIRED for §11.4.98 compliance)

Per Universal Constitution §11.4.98 + Herald §108.m (anchored 2026-05-28): every live test MUST be re-runnable end-to-end without manual intervention. The Telegram Bot API alone cannot satisfy this for inbound-driven flows (`TestSubscribe_LiveBotAPI`, `tests/test_wave6_live_loop.sh`, Wave 6.5 lifecycle scenarios) because **bots cannot see other bots' messages in non-DM contexts** — empirically verified 2026-05-28 against group `-4946584787`: `@pherald_qa_bot` (id 8971749017) sent msg_id=18, `@atmosphere_worker_bot` observed 0 updates. Telegram's privacy boundary is structural, not configurable.

Solution: drive QA tests from a **real Telegram user account** via the **MTProto protocol** (the same protocol Telegram apps use). The harness lives in `qaherald/internal/mtproto/` (vendor: `github.com/gotd/td`). A user account in the same chat as the bot can send messages that the bot's `getUpdates` poller picks up, enabling fully-autonomous closed-loop testing.

⚠️ CRITICAL — anti-ban hygiene (read BEFORE Step 1)

Per Telegram's official docs + gotd/td's "How to not get banned" guide: **all accounts that log in via unofficial Telegram clients are automatically put under observation**. To avoid permanent bans:

1. **EMAIL `recover@telegram.org` BEFORE or AT first login** declaring the userbot's purpose. Template in `docs/requirements/blockers/missing_env_variables.md` §"CRITICAL — read this BEFORE starting Step 1".
2. **DO NOT use VoIP / Google Voice / Twilio / TextNow numbers** — Telegram flags them aggressively. Use only your personal account OR a dedicated SIM/eSIM.
3. **One phone = ONE app_id, FOREVER** — if your phone already has an app at my.telegram.org/apps, you MUST reuse it; you cannot create a second one.
4. **app_id + app_hash cannot be regenerated** — they are permanently bound to the Telegram account. Treat them as immutable secrets.
5. Use the harness **passively** (receive more than send). Wire `github.com/gotd/contrib/middleware/ratelimit` + `floodwait` (already vendored in `submodules/gotd-td`).

If banned despite all this, email `recover@telegram.org` from the same address explaining the userbot's purpose. Automated false-positives are recoverable; deliberate abuse is not.

Variables (add to `.env`)

Variable	What	Example shape	Source
`HERALD_MTPROTO_APP_ID`	App api_id	`12345678` (integer, 5-8 digits)	https://my.telegram.org/apps
`HERALD_MTPROTO_APP_HASH`	App api_hash	32-char lowercase hex	

Variable	What	Example shape	Source
			https://my.telegram.org/apps
`HERALD_MTPROTO_PHONE`	QA-driver phone (E.164)	`+12025551234`	YOU choose — see "Account choice" below
`HERALD_MTPROTO_PASSWORD`	Cloud 2FA password	(32+ chars)	Set in Telegram → Settings → Privacy and Security → Two-Step Verification (only if 2FA enabled on the QA account)
`HERALD_MTPROTO_SESSION_FILE`	Persisted session path	`~/.config/herald/mtproto.session`	Optional — defaults to that path. Auto-created on first login.

Step 1: Create the QA Telegram app at my.telegram.org

1. Open https://my.telegram.org/auth in a browser.
2. Enter the **QA-driver phone** (the one you'll use for `HERALD_MTPROTO_PHONE` — see "Account choice" below for guidance).
3. Telegram sends a login code to that phone's Telegram app (NOT SMS this time — only the in-app code).
4. Enter the code at my.telegram.org/auth.
5. After login, navigate to https://my.telegram.org/apps.
6. **First-time:** click "Create new application". Fill in (values updated 2026-05-28 after operator reported `Incorrect app name` validation error):
   - **App title:** `Herald` (simplest; works almost always) — fallbacks: `HeraldQA` (camelCase) → `Herald Test Harness` → `Herald Tools` → `Herald Lab`. **AVOID bare acronyms like** `QA` — Telegram's classifier rejects them with `Incorrect app name`.
   - **Short name:** `heraldqa<random4>` (e.g. `heraldqa5kx9`) — **NO UNDERSCORES**. Must be 5-32 chars, STRICTLY alphanumeric [a-zA-Z0-9] — despite Telegram's hint reading "alphanumeric, 5-32 characters", underscores ARE REJECTED with `Incorrect app name` (operator-confirmed 2026-05-28: `herald_qa_5kx9` REJECTED, `heraldqa5kx9` ACCEPTED). GLOBALLY UNIQUE across all Telegram apps — plain `heraldqa` may already be taken.
   - **URL:** `https://herald.local` (or any valid `http(s)://` URL — Telegram requires the `http(s)://` prefix; bare domains rejected). Field is NOT actually optional.
   - **Platform:** `Desktop` (fallback if `Other` is rejected on your Telegram form version).
   - **Description:** `Herald automation harness for closed-loop testing.` (plain ASCII; no § symbol; under 200 chars; avoid acronyms QA/CI/CD).
7. Click "Create application". **If you get** `Incorrect app name` — the App title contains a bare acronym Telegram flags as non-app-like; try the fallback titles above in order.
8. Telegram displays:
   - `App api_id` — small integer (5-8 digits) → set as `HERALD_MTPROTO_APP_ID`
   - `App api_hash` — 32-char hex string → set as `HERALD_MTPROTO_APP_HASH`

9. **Copy both immediately.** my.telegram.org will not let you re-display the api_hash once you navigate away (you can revoke + recreate the app if lost, but existing sessions break).

Step 2: Account choice — which phone to use

The phone in `HERALD_MTPROTO_PHONE` MUST belong to a Telegram **user account** (not a bot). Three options:

Option	Pros	Cons
(a) Your personal Telegram account	Fastest setup. No SIM/VoIP needed.	Test runs send messages "from you" in the QA chat — visible to anyone in the group. Account-level mistakes (e.g. bug deletes all messages) hit your real account. Not recommended for production CI.
(b) Dedicated QA SIM	Clean separation. Account is purpose-built.	Requires a physical SIM and ~$10-20/month.
(c) Voice-over-IP (Google Voice / Twilio / TextNow) number	No physical SIM. Free or cheap.	Telegram increasingly rejects VoIP numbers — works ~60% of the time. Try first; fall back to (b).

Operator recommendation for first-cycle: use (a) your personal account. Once the harness is proven working, migrate to (b) for steady-state. The session file is portable — just `HERALD_MTPROTO_PHONE` + the session file change.

Step 3: Add the QA account to the chat

The QA user account MUST be a member of `HERALD_TGRAM_CHAT_ID`. If using (a) and you're already a member: done. If using (b)/(c):

1. Have an existing member of the group invite the new account (group invite link or add-by-phone).
2. Verify membership before running the test: log into Telegram as the QA account and confirm the group is visible.

Step 4: First-run login (one-time bootstrap — §11.4.98 permitted exception)

The harness binary (planned: `qaherald mtproto login`) connects to Telegram MTProto, sends a login code to the QA account's Telegram app, prompts you at the CLI to type the code (+ 2FA password if applicable), and persists the resulting session to `HERALD_MTPROTO_SESSION_FILE` (default `~/.config/herald/mtproto.session`). This is a **one-time interactive step**; subsequent test runs are fully autonomous (the §11.4.98 single permitted exception — configuration, not test driving).

Workflow (planned — implementation under Wave 8 Track B):

```
# After populating .env with the 3 MTProto vars:
set -a; source .env; set +a

# One-time interactive login:
qaherald mtproto login
#   → prompts: "Enter code Telegram sent to <phone>:"
```

```
#   → if 2FA enabled: "Enter cloud password:"
#   → on success: writes ~/.config/herald/mtproto.session
#   → prints: "MTProto session active for @<username>
        (user_id=<id>)"

# Subsequent test runs (FULLY AUTONOMOUS — no CLI prompts):
go test -tags=integration_mtproto -count=3 -run
        TestSubscribe_LiveMTProto \
    ./commons_messaging/channels/tgram/...
```

Step 5: Verify

A `qaherald mtproto whoami` command (planned) connects via the persisted session and prints the QA account's identity — proves the session is alive without sending any messages. Re-run before each CI campaign to confirm Telegram hasn't expired the session (sessions are valid indefinitely unless explicitly revoked from another device).

Security notes (composes with §11.4.10)

1. **Never commit** `HERALD_MTPROTO_APP_HASH` **or the session file to git.** Both are sufficient to impersonate the QA account.
2. **Never share the api_hash with another project.** Each project gets its own my.telegram.org app per Telegram's terms; sharing risks rate-limit bans across both.
3. **Treat the session file like an SSH private key.** `chmod 600`, owned by the running user, never world-readable.
4. **Rotate the api_hash + invalidate the session** if leaked: my.telegram.org/apps → revoke + recreate; then `rm ~/.config/herald/mtproto.session` and re-login.
5. **HRD-133 sanitizer applies to MTProto errors too** — the harness MUST `sanitizeMTProtoError()` wrap every error path to ensure api_hash, session bytes, and 2FA password text never appear in committed logs / `docs/qa/` transcripts.

Audit trail per §11.4.98

After Wave 8 Track B lands, every closed-loop run produces a transcript under `docs/qa/HRD-LIVE-MTPROTO-<TS>/` documenting:

- Tests executed (`TestSubscribe_LiveMTProto`, `TestWave6_LiveMTProto_ClosedLoop`, etc.)
- `-count=3` consecutive PASS proof per §11.4.98 rule (4)
- Self-cleaning state proof (chat-cleanup, session-file unchanged)
- Sanitizer audit (token-shape regex + api_hash-shape regex on all transcripts → 0 matches required)
- COMPLIANT classification per §108.m audit (release-gate item)

A `TestSubscribe_LiveMTProto` test that requires any human action during execution is a release-blocking defect. SKIP-with-reason (when credentials absent) is the §11.4.3 correct posture; SKIP-reported-as-PASS or stale-evidence is forbidden.

Troubleshooting

`AUTH_KEY_UNREGISTERED` **on first run**: The session file from a previous account is still present. `rm ~/.config/herald/mtproto.session` and re-run `qaherald mtproto login`.

`PHONE_CODE_INVALID`: The login code was entered wrong or expired (codes valid ~5 minutes). Re-run; Telegram sends a fresh one.

`SESSION_PASSWORD_NEEDED`: The QA account has 2FA enabled. Add `HERALD_MTPROTO_PASSWORD=<your cloud password>` to `.env` and re-run.

`FLOOD_WAIT_<N>`: Telegram is rate-limiting login attempts on this phone. Wait `<N>` seconds (could be hours during aggressive throttling).

`PHONE_NUMBER_INVALID`: Wrong E.164 format. Use `+<country code><number>` with no spaces or dashes (e.g. `+12025551234`).

`USER_DEACTIVATED`: Telegram has deactivated the QA account (often happens to VoIP / suspicious-pattern accounts). Use option (b) — dedicated SIM.

The harness sends messages but `@atmosphere_worker_bot`'s poller doesn't see them: Confirm the QA account is a member of `HERALD_TGRAM_CHAT_ID`. The privacy boundary that blocks bot-to-bot ALSO blocks "user-not-in-chat → bot". Add the QA account to the group.

The harness sees the worker bot's reply via `messages.getHistory` but it's not new: MTProto returns chat history; you need to filter by `date > test_start_time` to confirm the reply was sent during the test, not a stale message from a prior run. The harness `WaitForReply()` helper does this filtering — never roll your own.

Sources verified

Per HelixConstitution §11.4.99 + Herald §108.n (Latest-Source Documentation Cross-Reference Mandate). Every instruction in the MTProto section of this document was cross-referenced against the LATEST official online documentation of the relevant service before publication.

Last verified: 2026-05-28

Source	URL / path	Authored / verified
Telegram official API docs — "Obtaining an api_id"	https://core.telegram.org/api/obtaining_api_id	The one-phone-one-app-id constraint; the under-observation note on unofficial clients; the Terms-of-Service citation.
gotd/td library — "How to not get banned?" + MadelineProto wisdom	`submodules/gotd-td/.github/SUPPORT.md` (v0.144.0 @ commit `76282a6`)	The pre-login `recover@telegram.org` email step; the no-VoIP recommendation; the passive-use + rate-limit guidance.
Telegram official API docs — Bot API (HRD-011 /	https://core.telegram.org/bots/api	Bot token format, `/getMe`, `/sendMessage`,

Source	URL / path	Authored / verified
`@atmosphere_worker_bot` integration)		`getUpdates`, webhook secret_token validation.
Anthropic / Claude Code docs (HRD-012)	https://docs.anthropic.com/claude-code	`claude --resume <UUID>` session model; session UUID handling; PATH lookup defaults.
Postgres + Redis container conventions	`containers/` submodule + Herald spec V3 §9.3	Container ports (24100 / 24101); password-rotation flows; volume-data persistence.
Empirical operator testing 2026-05-28	`docs/qa/HRD-LIVE-20260528T082128Z/`	Bot-to-bot wall empirical proof; live HealthCheck + Send + Bootstrap PASS evidence.

Other sections of this document (Postgres / Redis / non-MTProto Telegram / Claude Code) were authored against the same canonical sources during their respective HRD lifecycles (HRD-010 / HRD-011 / HRD-012). They are due for §11.4.99 re-verification at the next Herald release boundary or sooner if any of those services publish a breaking change.

Re-verification cadence (per §11.4.99 (C)): Telegram-related sections (risk-classified per §11.4.99 (D)) → 90-day staleness, next due **2026-08-26**. Other sections → 6-month staleness, next due **2026-11-28**.